1. Write a program to Append 2 arrays.

Algorithm

Step 1) **Initialize**
Let Array1 'a' and Array2 'b' be the input arrays.
Create a new ResultArray 'c' of size n1 + n2.
Step 2) **Copy elements from Array1**:
For each i from 0 to n1 - 1, set c[i] = a[i].
Step 3)**Copy elements from Array2**:
For each j from 0 to n2 - 1, set c[n1 + j] = b[j].
Stept 4) **Print ResultArray 'c'**.

Code

```
#include<stdio.h>
int main()
{
int aSize, bSize, mSize, i, j;
int a[10], b[10], Merged[20];
printf("\n Please Enter the First Array Size : ");
scanf("%d", &aSize);
printf("\nPlease Enter the First Array Elements : ");
for(i = 0; i < aSize; i++)
 {
 scanf("%d", &a[i]);
 }
 printf("\n Please Enter the Second Array Size : ");
scanf("%d", &bSize);
printf("\nPlease Enter the Second Array Elements : ");
for(i = 0; i < bSize; i++)
 {
 scanf("%d", &b[i]);
 }

 for(i = 0; i < aSize; i++)
 {
 Merged[i] = a[i];
 }

mSize = aSize + bSize;
for(i = 0, j = aSize; j < mSize && i < bSize; i++, j++)
 {
 Merged[j] = b[i];
 }
printf("\n a[%d] Array Elements After Merging \n", mSize);
for(i = 0; i < mSize; i++)
 {
 printf(" %d \t ",Merged[i]);
 }
```

ANSHID BABU T M

```
 return 0;
}
```

2. Implement a Stack using an array.

Algorithm

1. Check if the stack is full.
   If(top==max-1)
2. If the stack is full, produce an overflow message and exit.
3. If the stack is not full, increments top to point next space.
   top=top+1
4. Adds data element to the stack location, where the top points.
   stack[top]=data
5. Returns success.
6. Check if the stack is empty
   If(top==-1)
7. if the stack is empty produce a message underflow and exit.
8. decrement the top, top=top-1
9. display the stack

Code

```
#include<stdio.h>
int stack[100],choice,n,top,x,i;
void push(void);
void pop(void);
void display(void);
int main()
{
 //clrscr();
 top=-1;
 printf("\n Enter the size of STACK[MAX=100]:");
 scanf("%d",&n);
 printf("\n\t STACK OPERATIONS USING ARRAY");
 printf("\n\t--------------------------------");
 printf("\n\t 1.PUSH\n\t 2.POP\n\t 3.DISPLAY\n\t 4.EXIT");
 do
 {
 printf("\n Enter the Choice:");
 scanf("%d",&choice);
 switch(choice)
 {
 case 1:
 push();
```

ANSHID BABU T M

```
 break;
 case 2:
 pop();
 break;
 case 3:
 display();
 break;

 case 4:
 printf("\n\t EXIT POINT ");
 break;
 default:

 printf ("\n\t Please Enter a Valid Choice(1/2/3/4)");
 }
 }
 while(choice!=4);
 return 0;
}
void push()
{
 if(top>=n-1)
 {
 printf("\n\tSTACK is over flow");

 }
 else
 {
 printf(" Enter a value to be pushed:");
 scanf("%d",&x);
 top++;
 stack[top]=x;
 }
}
void pop()
{
 if(top<=-1)
 {
 printf("\n\t Stack is under flow");
 }
 else
 {
 printf("\n\t The popped elements is %d",stack[top]);
 top--;
 }
}
void display()
{
 if(top>=0)
 {
```

ANSHID BABU T M

```
 printf("\n The elements in STACK \n");
 for(i=top; i>=0; i--)
 printf("\n%d",stack[i]);
 printf("\n Press Next Choice");
 }
 else
 {
 printf("\n The STACK is empty");
 }
}
```

3. Write a program to implement stack using LinkedList

Algorithm

Push operation

1. Initialise a node
2. Update the value of that node by data i.e. **node->data = data**
3. Now link this node to the top of the linked list
4. And update the top pointer to the current node **top=top->ptr;**

Pop operation

5. First Check whether there is any node present in the linked list or not, if not then return
   if (top == NULL)
   Stack is empty
6. Otherwise make a pointer let's say **temp** to the top node and move forward the top node by 1 step
7. Now free this temp node

Peek element

8. **top->info**

Display operation

9. print the value of node
   print top->info;
10. update pointer
    top1 = top->ptr;

code

```
#include <stdio.h>
#include <stdlib.h>

struct node

{
int info;
 struct node *ptr;
}*top,*top1,*temp;
```

ANSHID BABU T M

```
int topelement();
void push(int data);
void pop();
void empty();
void display();
void destroy();
void stack_count();
void create();
int count = 0;
void main()
{
 int no, ch, e;
 printf("\n 1 - Push");
 printf("\n 2 - Pop");
 printf("\n 3 - Top");
 printf("\n 4 - Empty");
 printf("\n 5 - Exit");
 printf("\n 6 - Dipslay");
 printf("\n 7 - Stack Count");
 printf("\n 8 - Destroy stack");
 create();
 while (1)
 {
 printf("\n Enter choice : ");
 scanf("%d", &ch);
 switch (ch)
 {
 case 1: printf("Enter data : ");
 scanf("%d", &no);
 push(no);
 break;
 case 2: pop();
 break;
 case 3: if (top == NULL)
 printf("No elements in stack");
 else
 {
 e = topelement();
 printf("\n Top element : %d", e);
 }
 break;
 case 4: empty();
 break;
 case 5: exit(0);
 case 6: display();
 break;
 case 7: stack_count();
 break;
 case 8: destroy();
 break;
```

ANSHID BABU T M

```
 default :
 printf(" Wrong choice, Please enter correct choice ");
 break;
 }
 }
}
void create()
{
 top = NULL;
}
void stack_count()
{
 printf("\n No. of elements in stack : %d", count);
}
void push(int data)
{
 if (top == NULL)
 {
 top =(struct node *)malloc(1*sizeof(struct node));
 top->ptr = NULL;
 top->info = data;
 }
 else
 {
 temp =(struct node *)malloc(1*sizeof(struct node));
 temp->ptr = top;
 temp->info = data;
 top = temp;
 }
 count++;
}
void display()
{
 top1 = top;
 if (top1 == NULL)
 {
 printf("Stack is empty");
 return;
 }
 while (top1 != NULL)
 {
 printf("%d ", top1->info);
 top1 = top1->ptr;
 }
}
void pop()
{
 top1 = top;
 if (top1 == NULL)
 {
```

ANSHID BABU T M

```
 printf("\n Error : Trying to pop from empty stack");
 return;
 }
 else
 top1 = top1->ptr;
 printf("\n Popped value : %d", top->info);
 free(top);
 top = top1;
 count--;
}
int topelement()
{
 return(top->info);
}
void empty()
{
 if (top == NULL)
 printf("\n Stack is empty");
 else
 printf("\n Stack is not empty with %d elements", count);
}
void destroy()
{
 top1 = top;
 while (top1 != NULL)
 {
 top1 = top->ptr;
 free(top);
 top = top1;
 top1 = top1->ptr;
 }
 free(top1);
 top = NULL;
 printf("\n All stack elements destroyed");
 count = 0;
}
```

ANSHID BABU T M

4. Implement a sparse matrix using an array

Algorithm

The triplet (row, column, value) format stores only the non-zero elements.

The row index of the non-zero element.
The column index of the non-zero element.
The non-zero element itself.
Steps:

1. Input the Matrix: Get the size and elements of the matrix.

2. Count Non-Zero Elements: Traverse the matrix to find the non-zero elements.
3. Create Triplet Representation: Store the row, column, and value of each non-zero element in an array.

```
 if (matrix[i][j] != 0) {
     sparse[nonZeroCount].row = i;
     sparse[nonZeroCount].col = j;
     sparse[nonZeroCount].value = matrix[i][j];
           nonZeroCount++;
         }
```

4. Output the Triplet Representation: Display the sparse matrix in triplet form.

```
#include <stdio.h>

#define MAX 100

Struct SparseMatrix
{
   int row;    // Row index
   int col;    // Column index
   int value;  // Value at (row, col)
};

int main() {
   int matrix[10][10];
   struct SparseMatrix sparse[MAX];
   int i, j, rows, cols, nonZeroCount = 0;

   printf("Enter the number of rows and columns of the matrix:\n");
   scanf("%d%d", &rows, &cols);

   printf("Enter the elements of the matrix:\n");
   for (i = 0; i < rows; i++) {
      for (j = 0; j < cols; j++) {
         scanf("%d", &matrix[i][j]);

         if (matrix[i][j] != 0) {
            sparse[nonZeroCount].row = i;
            sparse[nonZeroCount].col = j;
            sparse[nonZeroCount].value = matrix[i][j];
            nonZeroCount++;
         }
      }
   }
   printf("\nOriginal Matrix:\n");
   for (i = 0; i < rows; i++) {
      for (j = 0; j < cols; j++) {
         printf("%d ", matrix[i][j]);
      }
      printf("\n");
```

AWSHID BABU T M

```
    }

    printf("\nSparse Matrix Representation (Triplet Format):\n");
    printf("Row   Column   Value\n");
    for (i = 0; i < nonZeroCount; i++) {
        printf("%d      %d      %d\n", sparse[i].row, sparse[i].col, sparse[i].value);
    }

    return 0;
}
```

5. Implement a Queue using an array

Algorithm

1. Enqueue(x):
   Check if the queue is full (rear == MAX_SIZE - 1).
   If full, return "Queue Overflow".
   If the queue is empty, set both front and rear to 0.
   Otherwise, increment rear and insert x at queue[rear].
2. Dequeue():
   Check if the queue is empty (front == -1 or front > rear).
   If empty, return "Queue Underflow".
   Retrieve and return the element at queue[front].
   Increment front. If front exceeds rear, reset front and rear to -1.
3. Peek():
   Check if the queue is empty.
   If not, return the element at queue[front].
4. Empty():
   Returns True if front == -1.
5. Full():
   Returns True if rear == MAX_SIZE - 1.

Code

```
 #include<stdio.h>
#include<conio.h>
#define SIZE 10
void enQueue(int);
void deQueue();
void display();
int queue[SIZE], front = -1, rear = -1;
void main()
{
 int value, choice;
 clrscr();
 while(1){
 printf("\n\n***** MENU *****\n");
 printf("1. Insertion\n2. Deletion\n3. Display\n4. Exit");
 printf("\nEnter your choice: ");
```

ANSHID BABU T M

```
 scanf("%d",&choice);
 switch(choice){
case 1: printf("Enter the value to be insert: ");
scanf("%d",&value);
enQueue(value);
break;
case 2: deQueue();
break;
case 3: display();
break;
case 4: exit(0);
default: printf("\nWrong selection!!! Try again!!!");
 }
 }
}
void enQueue(int value){
 if(rear == SIZE-1)
 printf("\nQueue is Full!!! Insertion is not possible!!!");
 else{
 if(front == -1)
front = 0;
 rear++;
 queue[rear] = value;
 printf("\nInsertion success!!!");
 }
}
void deQueue(){
 if(front == rear)
 printf("\nQueue is Empty!!! Deletion is not possible!!!");
 else{
 printf("\nDeleted : %d", queue[front]);
 front++;
 if(front == rear)
front = rear = -1;
 }
}
void display()
{
 if(rear == -1)
 printf("\nQueue is Empty!!!");
 else{
 int i;
 printf("\nQueue elements are:\n");
 for(i=front; i<=rear; i++)
 printf("%d\t",queue[i]);
 }
        }
```

ANSHID BABU T M

6. Implement a Queue using a linked list

Algorithm

1. **Enqueue(x):**
   Create a new node with value x.
   If the queue is empty (front == NULL), set both front and rear to point to the new node.
   Otherwise, set rear->next to the new node, and update rear to point to the new node.
2. **Dequeue():**
   Check if the queue is empty (front == NULL).
   If empty, return "Queue Underflow".
   Otherwise, retrieve the value from the front node, move front to front->next.
   If after dequeueing the queue becomes empty (front == NULL), set rear = NULL.
3. **Peek():**
   - Return the value of the front node, if the queue is not empty.
4. **Empty():**
   - Returns True if front == NULL.
5. display the elements

Code

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for the linked list
struct Node {
    int data;
    struct Node* next;
};

// Queue structure
struct Queue {
    struct Node *front, *rear;
};

// Function to create a new node
struct Node* newNode(int data) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = data;
    temp->next = NULL;
    return temp;
}
```

ANSHID BABU T M

```
// Function to create a new queue
struct Queue* createQueue() {
    struct Queue* q = (struct Queue*)malloc(sizeof(struct Queue));
    q->front = q->rear = NULL;
    return q;
}

// Function to add an element to the queue (enqueue)
void enqueue(struct Queue* q, int data) {
    struct Node* temp = newNode(data);

    // If the queue is empty, the new node is both front and rear
    if (q->rear == NULL) {
        q->front = q->rear = temp;
        return;
    }

    // Add the new node at the end of the queue and update rear
    q->rear->next = temp;
    q->rear = temp;
}

// Function to remove an element from the queue (dequeue)
void dequeue(struct Queue* q) {
    // If the queue is empty, return
    if (q->front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    // Store the previous front and move the front pointer to the next node
    struct Node* temp = q->front;
    q->front = q->front->next;

    // If the front becomes NULL, then change rear to NULL
    if (q->front == NULL) {
        q->rear = NULL;
    }

    free(temp);
}

// Function to display the queue
void displayQueue(struct Queue* q) {
    if (q->front == NULL) {
        printf("Queue is empty\n");
        return;
    }
```

ANSHID BABU T M

```
    struct Node* temp = q->front;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Main function to test the queue
int main() {
    struct Queue* q = createQueue();

    enqueue(q, 10);
    enqueue(q, 20);
    enqueue(q, 30);

    printf("Queue after enqueue operations: ");
    displayQueue(q);

    dequeue(q);
    printf("Queue after dequeue operation: ");
    displayQueue(q);

    return 0;
        }
```

7. Create a singly LinkedList of n nodes and display it.

Algorithm:

1. Define a structure for the node with two fields: one for data and one for the pointer to the next node.
2. Initialize an empty list with head = NULL.
3. Create nodes dynamically using malloc() and add data into each node.
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
4. Link each new node to the previous node by setting the next pointer.
   head->next = second;
   second->next = third;
   third->next = fourth;
5. Display the list by traversing it from the head node to the last node.

Code

```
#include <stdio.h>
#include <stdlib.h>
// Define the structure of a node
struct Node {
```

ANSHID BABU T M

```
 int data;
 struct Node* next;
};
// Function to print the linked list
void printList(struct Node* head) {
 struct Node *p= head; // Start from the head
 while (p != NULL) {
 printf("%d -> ", p->data); // Print the data of the current node
 p= p->next; // Move to the next node
 }
 printf("NULL\n"); // End of the list
}
// Function to create a new node
struct Node* createNode(int data) {
 struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
 newNode->data = data;
 newNode->next = NULL;
 return newNode;
}
int main() {
 // Create nodes
 struct Node* head = createNode(1);
 struct Node* second = createNode(2);
 struct Node* third = createNode(3);
 struct Node* fourth = createNode(4);
 // Link nodes
 head->next = second;
 second->next = third;
 third->next = fourth;
 // Traverse and print the list
 printList(head);
 return 0;
}
```

8. Delete a given element from a singly linked list

Algorithm:

1. Define a structure for the node with two fields: data and the next pointer.
2. Input the position of the node to be deleted.
3. Handle edge cases:

   If the list is empty, print a message and return it.

   If the position is 1, delete the head node.

4. Traverse the list to find the node just before the one to be deleted.

ANSHID BABU T M

5. Change the **`next`** pointer of the previous node to skip over the node to be deleted.
6. Free the memory of the deleted node.
7. Display the modified list.

Code

```
#include <stdio.h>

#include <stdlib.h>

// Define a structure for the node

struct Node {

    int data;

    struct Node* next;

};

// Function to create a linked list with n nodes

struct Node* createLinkedList(int n) {

    struct Node *head = NULL, *temp = NULL, *newNode;

    int data, i;

    for(i = 1; i <= n; i++) {

        newNode = (struct Node*)malloc(sizeof(struct Node));

        printf("Enter data for node %d: ", i);

        scanf("%d", &data);

        newNode->data = data;

        newNode->next = NULL;

        if(head == NULL) {

            head = newNode;

        } else {

            temp->next = newNode;
```

ANSHID BABU T M

```
        }
        temp = newNode;
    }
    return head;
}
// Function to delete a node at a given position
struct Node* deleteNode(struct Node* head, int position) {
    struct Node* temp = head;
    struct Node* prev = NULL;
    int i;
    // Case 1: List is empty
    if(head == NULL) {
        printf("List is empty.\n");
        return head;
    }
    // Case 2: Deleting the head node (position = 1)
    if(position == 1) {
        head = temp->next;
        free(temp);  // Free the old head
        return head;
    }
    // Case 3: Deleting a node at a given position (not the head)
    for(i = 1; i < position && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
```

ANSHID BABU T M

```
    }
    // If the position is greater than the number of nodes
    if(temp == NULL) {
        printf("Position out of range.\n");
        return head;
    }
    // Unlink the node to be deleted and free it
    prev->next = temp->next;
    free(temp);
    return head;
}
// Function to display the linked list
void displayLinkedList(struct Node* head) {
    struct Node* temp = head;
    if(head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked list: ");
    while(temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

ANSHID BABU T M

```
int main() {
    int n, position;
    struct Node* head = NULL;
    // Input number of nodes
    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    // Create linked list
    head = createLinkedList(n);
    // Display the linked list
    displayLinkedList(head);

    // Input the position of the node to delete
    printf("Enter the position of the node to delete: ");
    scanf("%d", &position);
    // Delete the node at the given position
    head = deleteNode(head, position);
    // Display the updated linked list
    displayLinkedList(head);
    return 0;
}
```

ANSHID BABU T M

9.Sort a singly linked list.

Algorithm

1) **Input**: The number of nodes `n` and the values for each node.

2) **Create the linked list** by inserting nodes one by one at the end.
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   struct Node* last = *head;
   newNode->data = data;
   newNode->next = NULL;

3) **Print the unsorted linked list**.
4) **Sort the linked list** using the bubble sort algorithm:
   tempData = ptr1->data;
   ptr1->data = ptr1->next->data;
   ptr1->next->data = tempData;

5) Compare adjacent nodes and swap their data if needed.
6) Repeat this process until no more swaps are needed (i.e., the list is sorted).
7) **Print the sorted linked list**.
8) **End**.

```
#include <stdio.h>

#include <stdlib.h>


// Define the structure for a node

struct Node {

    int data;

    struct Node* next;

};


// Function to insert a new node at the end of the linked list

void insertNode(struct Node** head, int data) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

ANSHID BABU T M

```
    struct Node* last = *head;
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    while (last->next != NULL) {
        last = last->next;
    }
    last->next = newNode;
}

// Function to print the linked list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

ANSHID BABU T M

```
// Function to sort the linked list using bubble sort
void sortList(struct Node* head) {
    int swapped, tempData;
    struct Node *ptr1;
    struct Node *lptr = NULL;

    if (head == NULL)
        return;

    do {
        swapped = 0;
        ptr1 = head;

        while (ptr1->next != lptr) {
            if (ptr1->data > ptr1->next->data) {
                // Swap the data
                tempData = ptr1->data;
                ptr1->data = ptr1->next->data;
                ptr1->next->data = tempData;
                swapped = 1;
            }
            ptr1 = ptr1->next;
        }
        lptr = ptr1;
    } while (swapped);
```

ANSHID BABU T M

```
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, i;

    // Accept the number of nodes
    printf("Enter the number of nodes: ");
    scanf("%d", &n);

    // Accept the values from the user
    for (i = 0; i < n; i++) {
        printf("Enter value for node %d: ", i + 1);
        scanf("%d", &value);
        insertNode(&head, value);
    }

    // Display the list before sorting
    printf("Linked list before sorting:\n");
    printList(head);

    // Sort the linked list
    sortList(head);
```

ANSHID BABU T M

```
    // Display the list after sorting

    printf("Linked list after sorting:\n");

    printList(head);



    return 0;

}
```

10.Create a linked list and search an element from that list.

Algorithm

1) **Input**: The number of nodes `n` and the values for each node.
2) **Create a Linked List**: Insert the nodes one by one at the end using the `insertNode` function.
3) **Print the List**: Display the linked list before searching.
4) **Search for an Element**:
5) Accept the search value from the user.
6) Traverse the linked list using the `searchElement` function to check if the search value exists in any node.

   ```
   while (current != NULL) {

         if (current->data == value)
   ```

7) **Output the Result**:
8) If the value is found, print "Element found".
9) If the value is not found, print "Element not found".
10) **End**.

```
#include <stdio.h>

#include <stdlib.h>



// Define the structure for a node
```

ANSHID BABU TM

```
struct Node {
    int data;
    struct Node* next;
};

// Function to insert a new node at the end of the linked list
void insertNode(struct Node** head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* last = *head;
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    while (last->next != NULL) {
        last = last->next;
    }
    last->next = newNode;
}

// Function to search for an element in the linked list
int searchElement(struct Node* head, int value) {
```

ANSHID BABU TM

```
    struct Node* current = head;
    while (current != NULL) {
        if (current->data == value) {
            return 1; // Element found
        }
        current = current->next;
    }
    return 0; // Element not found
}

// Function to print the linked list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, searchVal, i;
```

ANSHID BABU T M

```
// Accept the number of nodes
printf("Enter the number of nodes: ");
scanf("%d", &n);

// Accept the values from the user
for (i = 0; i < n; i++) {
    printf("Enter value for node %d: ", i + 1);
    scanf("%d", &value);
    insertNode(&head, value);
}

// Display the list
printf("Linked list:\n");
printList(head);

// Accept the value to search
printf("Enter value to search in the list: ");
scanf("%d", &searchVal);

// Search the element in the list
if (searchElement(head, searchVal)) {
    printf("Element %d found in the list.\n", searchVal);
} else {
    printf("Element %d not found in the list.\n", searchVal);
}
```

ANSHID BABU T M

```
    return 0;
}
```

11. Sort a given list of strings

Algorithm

1. Start
2. Input number of strings n
3. For i from 0 to n-1:
4. Read the string into str[i]
5. If the string contains a newline character at the end (from fgets), replace it with null (\0) to remove the newline
6. Repeat for i = 0 to n-2:
7. Repeat for j = i+1 to n-1:
    a. Compare strings str[i] and str[j] character-by-character:
        i. Start from the first character
        ii. Continue while characters are equal
        iii. Stop at the first mismatch or at null (\0)
        iv. If str[i] > str[j] (lexicographically), then:
            1. Swap the strings using a temporary array temp[]
8. Display the sorted list of strings
9. End

```
#include <stdio.h>

#define MAX 100
#define SIZE 10

// Function to compare two strings manually
int stringCompare(char str1[], char str2[]) {
    int i = 0;
    while (str1[i] == str2[i]) {
        if (str1[i] == '\0')
            return 0; // Strings are equal
        i++;
    }
    return (str1[i] - str2[i]);
}

// Function to copy one string to another manually
void stringCopy(char dest[], char src[]) {
    int i = 0;
    while ((dest[i] = src[i]) != '\0') {
        i++;
```

ANSHID BABU TM

```
    }
}

// Main function to sort strings
int main() {
    char str[SIZE][MAX], temp[MAX];
    int i, j, n;

    printf("Enter number of strings: ");
    scanf("%d", &n);
    getchar(); // To consume the newline character

    printf("Enter %d strings:\n", n);
    for (i = 0; i < n; i++) {
        fgets(str[i], MAX, stdin);
        // Remove newline if present
        int k = 0;
        while (str[i][k] != '\0') {
            if (str[i][k] == '\n') {
                str[i][k] = '\0';
                break;
            }
            k++;
        }
    }

    // Bubble sort
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (stringCompare(str[i], str[j]) > 0) {
                stringCopy(temp, str[i]);
                stringCopy(str[i], str[j]);
                stringCopy(str[j], temp);
            }
        }
    }

    printf("\nSorted Strings:\n");
    for (i = 0; i < n; i++) {
        printf("%s\n", str[i]);
    }

    return 0;
}
```

Input:

Enter number of strings: 4
zebra
apple
banana
grape

ANSHID BABU T M

Output:
Sorted Strings:
apple
banana
grape
zebra

12. Implement Pattern matching algorithm.

Naive Pattern Matching Algorithm

1. **Start**
2. **Input** the text `T` and the pattern `P`
3. Let `n` = length of `T`, and `m` = length of `P`
4. **Repeat for i = 0 to (n - m)**:
   - Initialize a variable `match = true`
   - For each character `j = 0 to (m - 1)`:
     - If `T[i + j] != P[j]`, then:
       - Set `match = false`
       - Break the inner loop
   - If `match == true`, then:
     - Report that the pattern is found at position `i`
5. **End**

Code

```
#include <stdio.h>

// Function to implement naive pattern matching
void patternMatching(char text[], char pattern[]) {
    int i, j;
    int textLen = 0, patternLen = 0;

    // Calculate length of text
    while (text[textLen] != '\0') {
        textLen++;
    }

    // Calculate length of pattern
    while (pattern[patternLen] != '\0') {
        patternLen++;
    }

    int found = 0;
    for (i = 0; i <= textLen - patternLen; i++) {
        for (j = 0; j < patternLen; j++) {
            if (text[i + j] != pattern[j]) {
```

ANSHID BABU T M

```
                break;
            }
        }

        if (j == patternLen) {
            printf("Pattern found at index %d\n", i);
            found = 1;
        }
    }

    if (!found) {
        printf("Pattern not found\n");
    }
}

int main() {
    char text[100], pattern[50];

    printf("Enter the text: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter the pattern: ");
    fgets(pattern, sizeof(pattern), stdin);

    // Removing newline characters from text and pattern
    int i = 0;
    while (text[i] != '\0') {
        if (text[i] == '\n') {
            text[i] = '\0';
            break;
        }
        i++;
    }

    i = 0;
    while (pattern[i] != '\0') {
        if (pattern[i] == '\n') {
            pattern[i] = '\0';
            break;
        }
        i++;
    }

    patternMatching(text, pattern);

    return 0;
}
```

Enter the text: ababcabcabababd
Enter the pattern: ababd

ANSHID BABU T M

Output
Pattern found at index 10

13. Implement a circular linked list.

1. Define Node Structure
   - Each node has:
     - data (integer)
     - next (pointer to next node)

---

2. Insertion at End
   1. Create a new node
   2. Assign data to the new node
   3. If the list is empty:
      - Set newNode->next = newNode
      - Set head = newNode
   4. Else:
      - Traverse to the last node (temp) such that temp->next == head
      - Set temp->next = newNode
      - Set newNode->next = head

---

3. Display List
   1. If list is empty, print "List is empty"
   2. Else:
      - Start from head
      - Traverse and print data of each node until you reach back to head

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

// Function to insert at end
void insertEnd(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        newNode->next = head; // Circular link
    } else {
        struct Node* temp = head;
```

ANSHID BABU T M

```
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
    }
}

// Function to display circular linked list
void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = head;
    printf("Circular Linked List: ");
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("\n");
}

// Main function
int main() {
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);
    insertEnd(40);

    display();

    return 0;
}
```

Circular Linked List: 10 20 30 40

14. Merge two sorted array into one sorted array.

Algorithm

1. Initialize three pointers i = 0, j = 0, k = 0
2. Compare elements of both arrays:
3. If A[i] < B[j], add A[i] to result, increment i
4. Else, add B[j] to result, increment j
5. If one array is exhausted, copy the remaining elements of the other array
6. Output the merged sorted array

ANSHID BABU T M

```
#include <stdio.h>

void mergeSortedArrays(int A[], int n, int B[], int m, int merged[]) {
    int i = 0, j = 0, k = 0;

    // Merge arrays while both have elements
    while (i < n && j < m) {
        if (A[i] < B[j]) {
            merged[k++] = A[i++];
        } else {
            merged[k++] = B[j++];
        }
    }

    // Copy remaining elements of A[]
    while (i < n) {
        merged[k++] = A[i++];
    }

    // Copy remaining elements of B[]
    while (j < m) {
        merged[k++] = B[j++];
    }
}

int main() {
    int A[100], B[100], merged[200];
    int n, m, i;

    printf("Enter the number of elements in first sorted array: ");
    scanf("%d", &n);
    printf("Enter %d sorted elements: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    printf("Enter the number of elements in second sorted array: ");
    scanf("%d", &m);
    printf("Enter %d sorted elements: ", m);
    for (i = 0; i < m; i++) {
        scanf("%d", &B[i]);
    }

    mergeSortedArrays(A, n, B, m, merged);

    printf("Merged Sorted Array: ");
    for (i = 0; i < n + m; i++) {
        printf("%d ", merged[i]);
    }
```

ANSHID BABU T M

```
    printf("\n");

    return 0;
}
```

First array: 1 3 5 7
Second array: 2 4 6 8

Output
Merged Sorted Array: 1 2 3 4 5 6 7 8

15. Search an element in the array using iterative binary search.

Algorithm: Iterative Binary Search

1. **Start**
2. **Initialize**:
   - `low = 0`
   - `high = n - 1`
3. **While low <= high**:
   - Calculate the middle index: `mid = (low + high) / 2`
4. **Compare** `arr[mid]` with `x`:
   - If `arr[mid] == x`, return the index `mid` (element found)
   - If `arr[mid] < x`, set `low = mid + 1` (ignore the left half)
   - If `arr[mid] > x`, set `high = mid - 1` (ignore the right half)
5. **If element is not found** after the loop ends, return `-1` (element not found)
6. **End**

```
#include <stdio.h>

// Function for Iterative Binary Search
int binarySearch(int arr[], int n, int x) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;  // To avoid overflow

        // Check if the element is present at mid
        if (arr[mid] == x) {
            return mid;
        }

        // If element is greater, ignore the left half
        if (arr[mid] < x) {
            low = mid + 1;
        }
        // If element is smaller, ignore the right half
        else {
```

ANSHID BABU T M

```
            high = mid - 1;
        }
    }

    return -1;  // Element not found
}

int main() {
    int arr[] = {2, 3, 4, 10, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;

    int result = binarySearch(arr, n, x);

    if (result != -1) {
        printf("Element found at index %d\n", result);
    } else {
        printf("Element not found\n");
    }

    return 0;
        }
```

16. Search an element in the array using recursive binary search.

Algorithm: Recursive Binary Search

1. **Start**
2. **Base Case**:
   - If `low > high`, return `-1` (element not found)
3. **Recursive Case**:
   - Calculate the middle index: `mid = low + (high - low) / 2`
   - If `arr[mid] == x`, return the index `mid` (element found)
   - If `arr[mid] < x`, recursively search the right half: `binarySearch(arr, mid + 1, high, x)`
   - If `arr[mid] > x`, recursively search the left half: `binarySearch(arr, low, mid - 1, x)`
4. **End**

```
#include <stdio.h>

// Function for Recursive Binary Search
int binarySearch(int arr[], int low, int high, int x) {
    // Base case: element is not found
    if (low > high) {
        return -1;
    }
```

ANSHID BABU T M

```
    int mid = low + (high - low) / 2;  // To avoid overflow

    // Check if the element is at mid
    if (arr[mid] == x) {
        return mid;
    }

    // If element is greater, search in the right half
    if (arr[mid] < x) {
        return binarySearch(arr, mid + 1, high, x);
    }

    // If element is smaller, search in the left half
    return binarySearch(arr, low, mid - 1, x);
}

int main() {
    int arr[] = {2, 3, 4, 10, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;

    int result = binarySearch(arr, 0, n - 1, x);

    if (result != -1) {
        printf("Element found at index %d\n", result);
    } else {
        printf("Element not found\n");
    }

    return 0;
}
```

17. Implement polynomial using linked list

1. Create nodes for each term of the polynomial.
2. Traverse both polynomials:
3. If the exponents of the terms are the same, add the coefficients and create a new node for the resulting term.
4. If the exponents differ, add the node with the term having the larger exponent to the result list.
5. Handle remaining terms in either of the polynomials when the end of one is reached.
6. Return the result polynomial

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a node in the polynomial linked list
struct Node {
    int coefficient;
```

ANSHID BABU T M

```
    int exponent;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int coeff, int exp) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->coefficient = coeff;
    newNode->exponent = exp;
    newNode->next = NULL;
    return newNode;
}

// Function to add two polynomials
struct Node* addPolynomials(struct Node* poly1, struct Node* poly2) {
    struct Node* result = NULL;  // Resultant polynomial
    struct Node* temp = NULL;

    // Traverse both polynomials
    while (poly1 != NULL && poly2 != NULL) {
        // If exponents are equal, add the coefficients
        if (poly1->exponent == poly2->exponent) {
            int coeffSum = poly1->coefficient + poly2->coefficient;
            if (coeffSum != 0) {  // Only add non-zero terms
                if (result == NULL) {
                    result = createNode(coeffSum, poly1->exponent);
                    temp = result;
                } else {
                    temp->next = createNode(coeffSum, poly1->exponent);
                    temp = temp->next;
                }
            }
            poly1 = poly1->next;
            poly2 = poly2->next;
        }
        // If exponent of poly1 is greater, add poly1's term
        else if (poly1->exponent > poly2->exponent) {
            if (result == NULL) {
                result = createNode(poly1->coefficient, poly1->exponent);
                temp = result;
            } else {
                temp->next = createNode(poly1->coefficient, poly1->exponent);
                temp = temp->next;
            }
            poly1 = poly1->next;
        }
        // If exponent of poly2 is greater, add poly2's term
        else {
            if (result == NULL) {
                result = createNode(poly2->coefficient, poly2->exponent);
```

ANSHID BABU TM

```
            temp = result;
        } else {
            temp->next = createNode(poly2->coefficient, poly2->exponent);
            temp = temp->next;
        }
        poly2 = poly2->next;
    }
}

// Add remaining terms from poly1
while (poly1 != NULL) {
    temp->next = createNode(poly1->coefficient, poly1->exponent);
    temp = temp->next;
    poly1 = poly1->next;
}

// Add remaining terms from poly2
while (poly2 != NULL) {
    temp->next = createNode(poly2->coefficient, poly2->exponent);
    temp = temp->next;
    poly2 = poly2->next;
}

return result;
}

// Function to display the polynomial
void displayPolynomial(struct Node* poly) {
    while (poly != NULL) {
        printf("%dx^%d", poly->coefficient, poly->exponent);
        poly = poly->next;
        if (poly != NULL) {
            printf(" + ");
        }
    }
    printf("\n");
}

int main() {
    // Create two polynomials: P(x) = 5x^3 + 3x^2 + 2x + 1 and Q(x) = 3x^2 + x + 4

    struct Node* poly1 = createNode(5, 3);
    poly1->next = createNode(3, 2);
    poly1->next->next = createNode(2, 1);
    poly1->next->next->next = createNode(1, 0);

    struct Node* poly2 = createNode(3, 2);
    poly2->next = createNode(1, 1);
    poly2->next->next = createNode(4, 0);
```

ANSHID BABU T M

```
    // Display the original polynomials
    printf("Polynomial 1: ");
    displayPolynomial(poly1);

    printf("Polynomial 2: ");
    displayPolynomial(poly2);

    // Add the polynomials
    struct Node* result = addPolynomials(poly1, poly2);

    // Display the result
    printf("\nSum of the polynomials: ");
    displayPolynomial(result);

    return 0;
}
```

Output

Polynomial 1: 5x^3 + 3x^2 + 2x^1 + 1x^0
Polynomial 2: 3x^2 + 1x^1 + 4x^0

Sum of the polynomials: 5x^3 + 6x^2 + 3x^1 + 5x^0

18. Implement bubble sort

Algorithm:

1. **Start at the beginning** of the array.
2. **Compare adjacent elements**. If the current element is greater than the next element, **swap** them.
3. **Repeat the process** for the entire array.
4. After each iteration, the largest element "bubbles up" to its correct position.
5. **Repeat the process** for the remaining unsorted elements, reducing the range by one each time.
6. The algorithm terminates when no more swaps are needed, indicating that the array is sorted.

```
#include <stdio.h>

#define SIZE 10

// Function to implement bubble sort
void bubbleSort(int arr[], int n) {
    int i, j, temp;
    // Outer loop for each pass
    for (i = 0; i < n - 1; i++) {
        // Inner loop to compare adjacent elements
        for (j = 0; j < n - i - 1; j++) {
```

ANSHID BABU T M

```
            // If current element is greater than next element, swap them
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Function to display the array
void displayArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[SIZE], n, i;

    // Input the size of the array
    printf("Enter number of elements: ");
    scanf("%d", &n);

    // Input elements in the array
    printf("Enter %d elements: \n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Display the original array
    printf("Original Array: ");
    displayArray(arr, n);

    // Perform bubble sort
    bubbleSort(arr, n);

    // Display the sorted array
    printf("Sorted Array: ");
    displayArray(arr, n);

    return 0;
}
```

Output

```
Enter number of elements: 5
Enter 5 elements:
```

ANSHID BABU T M

64 34 25 12 22
Original Array: 64 34 25 12 22
Sorted Array: 12 22 25 34 64

19. implement merge sort

Merge Sort Algorithm:

1. **Divide**:
   - If the array has only one element, return (as an array with a single element is trivially sorted).
   - Otherwise, split the array into two halves:
     - Find the midpoint `mid` of the array. If the array has indices from `left` to `right`, calculate:

       ```
       sql
       CopyEdit
       mid = left + (right - left) / 2
       ```

     - Recursively apply merge sort to both halves: `arr[left..mid]` and `arr[mid+1..right]`.
2. **Conquer**:
   - The recursion proceeds by breaking down the array into smaller and smaller subarrays until you reach base cases where each subarray has a size of one. A single element is already sorted.
3. **Merge**:
   - Once the subarrays are sorted (either one element or recursively sorted smaller arrays), merge the two sorted subarrays:
     - Create two temporary arrays `L[]` and `R[]` to hold the left and right subarrays.
     - Compare the elements of `L[]` and `R[]` one by one. Place the smaller element into the original array.
     - If there are any remaining elements in `L[]` or `R[]`, copy them into the original array.
   - This merging process ensures that the two subarrays are combined into a single sorted array.
4. **Repeat**:
   - The merge operation is performed recursively for the entire array, progressively merging smaller sorted arrays into larger sorted arrays until the entire array is sorted.

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

// Function to merge two halves of an array
```

ANSHIDABU TM

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Temporary arrays to hold the two halves
    int L[n1], R[n2];

    // Copy data to temporary arrays L[] and R[]
    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[mid + 1 + j];
    }

    // Merge the temporary arrays back into arr[left..right]
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[], if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[], if any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// Function to implement merge sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2; // Find the midpoint

        // Sort first and second halves
        mergeSort(arr, left, mid);
```

ANSHID BABU T M

```
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Function to display the array
void displayArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[SIZE], n;

    // Input the size of the array
    printf("Enter number of elements: ");
    scanf("%d", &n);

    // Input elements in the array
    printf("Enter %d elements: \n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Display the original array
    printf("Original Array: ");
    displayArray(arr, n);

    // Perform merge sort
    mergeSort(arr, 0, n - 1);

    // Display the sorted array
    printf("Sorted Array: ");
    displayArray(arr, n);

    return 0;
}
```

```
Enter number of elements: 5
Enter 5 elements:
64 34 25 12 22
Original Array: 64 34 25 12 22
Sorted Array: 12 22 25 34 64
```

ANSHID BABU T M

20. Implement linear sort

Steps of the Algorithm:

1. **Find the range** of the elements (i.e., the minimum and maximum values).
2. **Create a count array** to store the frequency of each element.
3. **Fill the count array** by counting how many times each element appears in the input array.
4. **Modify the count array** to store the cumulative sum of the counts (this will help to determine the position of each element).
5. **Place the elements** in their correct positions in the output array using the count array.
6. **Copy the sorted elements back** into the original array.

```
#include <stdio.h>

void countingSort(int arr[], int n) {
    int i, j;

    // Find the maximum and minimum element in the array
    int max = arr[0], min = arr[0];
    for (i = 1; i < n; i++) {
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }

    // Range of the elements (max - min + 1)
    int range = max - min + 1;

    // Create a count array and initialize all elements to 0
    int count[range];
    for (i = 0; i < range; i++) {
        count[i] = 0;
    }

    // Store the frequency of each element in the count array
    for (i = 0; i < n; i++) {
        count[arr[i] - min]++;
    }

    // Modify the count array to store cumulative count
    for (i = 1; i < range; i++) {
        count[i] += count[i - 1];
    }

    // Create an output array to store sorted elements
    int output[n];

    // Place the elements in the correct position in the output array
    for (i = n - 1; i >= 0; i--) {
```

ANSHID BABU T M

```
        output[count[arr[i] - min] - 1] = arr[i];
        count[arr[i] - min]--;
    }

    // Copy the sorted elements into the original array
    for (i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}

int main() {
    int arr[] = {4, 2, 2, 8, 3, 3, 1};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original Array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    countingSort(arr, n);

    printf("\nSorted Array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

Original Array:
4 2 2 8 3 3 1
Sorted Array:
1 2 2 3 3 4 8

21. Search an element in the 2-dimensional array

Algorithm

1. **Iterate through each row**: Start with the first row, then move to the next row, and so on.
2. **Iterate through each column**: For each row, check each element in the column.
3. **Check for the target value**: If the target element is found, return the position (row and column index).
4. **If the element is not found** after iterating through the entire array, return a failure or indicate that the element is not present.

```
#include <stdio.h>

// Function to search an element in a 2D array
int searchElement(int arr[][3], int rows, int cols, int target) {
```

ANSHID BABU T M

```
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (arr[i][j] == target) {
                printf("Element %d found at position (%d, %d)\n", target, i, j);
                return 1; // Return 1 to indicate element is found
            }
        }
    }
    printf("Element %d not found\n", target);
    return 0; // Return 0 if element is not found
}

int main() {
    // Example 2D array
    int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int target;

    printf("Enter the element to search: ");
    scanf("%d", &target);

    // Call the search function
    searchElement(arr, 3, 3, target);

    return 0;
}
```

Enter the element to search: 5
Element 5 found at position (1, 1)

22. Infix expression into its postfix expression

Steps for Conversion:

1. **Initialize**:
   - A stack to store operators and parentheses.
   - An empty list to store the resulting postfix expression.
2. **Process the Infix Expression**:
   - Read the infix expression from left to right.
   - For each character:
     - If it's an **operand**, add it to the result list.
     - If it's a **left parenthesis** (, push it onto the stack.
     - If it's a **right parenthesis**), pop operators from the stack and add them to the result until a left parenthesis (is encountered.
     - If it's an **operator**, pop operators from the stack to the result while the operator at the top of the stack has higher or equal precedence, then push the current operator onto the stack.
3. **End of Expression**:
   - After reading the entire infix expression, pop any remaining operators from the stack and add them to the result.

ANSHID BABU T M

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100

// Function to get the precedence of operators
int precedence(char c) {
    if (c == '+' || c == '-') {
        return 1;
    } else if (c == '*' || c == '/') {
        return 2;
    } else if (c == '^') {
        return 3;
    }
    return 0;
}

// Function to check if the character is an operand (number or variable)
int isOperand(char c) {
    return isalpha(c) || isdigit(c);
}

// Function to convert infix expression to postfix expression
void infixToPostfix(char* infix, char* postfix) {
    char stack[MAX];
    int top = -1;
    int j = 0;

    for (int i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];

        // If the character is an operand, add it to the result
        if (isOperand(c)) {
            postfix[j++] = c;
        }
        // If the character is '(', push it to the stack
        else if (c == '(') {
            stack[++top] = c;
        }
        // If the character is ')', pop until '(' is encountered
        else if (c == ')') {
            while (top != -1 && stack[top] != '(') {
                postfix[j++] = stack[top--];
            }
            top--; // Pop '(' from the stack
        }
        // If the character is an operator
        else {
            while (top != -1 && precedence(stack[top]) >= precedence(c)) {
```

AWSHID BABU T M

```
            postfix[j++] = stack[top--];
        }
        stack[++top] = c; // Push the current operator to the stack
    }
  }

  // Pop all remaining operators from the stack
  while (top != -1) {
    postfix[j++] = stack[top--];
  }

  postfix[j] = '\0'; // Null-terminate the postfix expression
}

int main() {
  char infix[MAX], postfix[MAX];

  printf("Enter the infix expression: ");
  scanf("%s", infix);

  infixToPostfix(infix, postfix);

  printf("Postfix expression: %s\n", postfix);

  return 0;
}
```

Output

Enter the infix expression: A+B*(C^D-E)
Postfix expression: ABCD^E-*+

23. Addition of 2 polynomials

Steps to Add Two Polynomials:

1. **Create a Linked List**: Each node contains a term of the polynomial, which consists of a coefficient and an exponent.
2. **Traverse both polynomials**: Start from the head of both linked lists and compare the exponents of the terms.
3. **Addition Logic**:
   - If the exponents are equal, add the coefficients of the terms and create a new node.
   - If the exponent of one term is greater, add the term with the larger exponent directly to the result list.
4. **Handle Remaining Terms**: After one of the lists is exhausted, directly append the remaining terms from the other list.

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a polynomial node
```

ANSHID BABU T M

```
struct Node {
    int coefficient;
    int exponent;
    struct Node *next;
};

// Function to create a new node
struct Node* createNode(int coefficient, int exponent) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->coefficient = coefficient;
    newNode->exponent = exponent;
    newNode->next = NULL;
    return newNode;
}

// Function to add two polynomials
struct Node* addPolynomials(struct Node* poly1, struct Node* poly2) {
    struct Node* result = NULL;
    struct Node* temp = NULL;

    // Traverse both polynomials
    while (poly1 != NULL && poly2 != NULL) {
        if (poly1->exponent == poly2->exponent) {
            // If exponents are same, add the coefficients
            int sum = poly1->coefficient + poly2->coefficient;
            struct Node* newNode = createNode(sum, poly1->exponent);
            if (result == NULL) {
                result = newNode;
            } else {
                temp->next = newNode;
            }
            temp = newNode;
            poly1 = poly1->next;
            poly2 = poly2->next;
        } else if (poly1->exponent > poly2->exponent) {
            // If exponent of poly1 is greater, add poly1's term to result
            struct Node* newNode = createNode(poly1->coefficient, poly1->exponent);
            if (result == NULL) {
                result = newNode;
            } else {
                temp->next = newNode;
            }
            temp = newNode;
            poly1 = poly1->next;
        } else {
            // If exponent of poly2 is greater, add poly2's term to result
            struct Node* newNode = createNode(poly2->coefficient, poly2->exponent);
            if (result == NULL) {
                result = newNode;
            } else {
```

AASHID BABU T M

```
            temp->next = newNode;
        }
        temp = newNode;
        poly2 = poly2->next;
    }
}

// If poly1 has remaining terms
while (poly1 != NULL) {
    struct Node* newNode = createNode(poly1->coefficient, poly1->exponent);
    if (result == NULL) {
        result = newNode;
    } else {
        temp->next = newNode;
    }
    temp = newNode;
    poly1 = poly1->next;
}

// If poly2 has remaining terms
while (poly2 != NULL) {
    struct Node* newNode = createNode(poly2->coefficient, poly2->exponent);
    if (result == NULL) {
        result = newNode;
    } else {
        temp->next = newNode;
    }
    temp = newNode;
    poly2 = poly2->next;
}

return result;
}

// Function to print a polynomial
void printPolynomial(struct Node* poly) {
    while (poly != NULL) {
        if (poly->next != NULL) {
            printf("%dx^%d + ", poly->coefficient, poly->exponent);
        } else {
            printf("%dx^%d", poly->coefficient, poly->exponent);
        }
        poly = poly->next;
    }
    printf("\n");
}

// Main function
int main() {
    // Create the first polynomial: 5x^4 + 3x^3 + 2x + 7
```

ANSHID BABU T M

```
    struct Node* poly1 = createNode(5, 4);
    poly1->next = createNode(3, 3);
    poly1->next->next = createNode(2, 1);
    poly1->next->next->next = createNode(7, 0);

    // Create the second polynomial: 4x^3 + 6x^2 + 3x + 8
    struct Node* poly2 = createNode(6, 2);
    poly2->next = createNode(4, 3);
    poly2->next->next = createNode(3, 1);
    poly2->next->next->next = createNode(8, 0);

    // Print the input polynomials
    printf("Polynomial 1: ");
    printPolynomial(poly1);
    printf("Polynomial 2: ");
    printPolynomial(poly2);

    // Add the two polynomials
    struct Node* result = addPolynomials(poly1, poly2);

    // Print the result of the addition
    printf("Resultant Polynomial: ");
    printPolynomial(result);

    return 0;
}
```

Polynomial 1: 5x^4 + 3x^3 + 2x^1 + 7x^0
Polynomial 2: 4x^3 + 6x^2 + 3x^1 + 8x^0
Resultant Polynomial: 5x^4 + 7x^3 + 6x^2 + 5x^1 + 15x^0

24. Implement a doubly linked list of integers

Algorithm for Doubly Linked List Operations
1. Creating a Node
Input: Integer data. Output: A new node with the given data.
Steps:
1. Allocate memory for a new node.
2. Set the data of the node to the given value.
3. Set both next and prev pointers of the node to NULL.
4. Return the new node.

2. Insert at the Beginning of the List
Input: Pointer to the head of the list, integer data. Output: List with a new node inserted at the beginning.
Steps:
1. Create a new node with the given data.
2. If the list is empty (head is NULL), set the head to the new node.
3. If the list is not empty, set the next pointer of the new node to the current head.
4. Set the prev pointer of the current head to the new node.

ANSHID BABU T M

5. Set the head of the list to the new node.

3. Insert at the End of the List

Input: Pointer to the head of the list, integer data. Output: List with a new node inserted at the end.

Steps:

1. Create a new node with the given data.
2. If the list is empty (head is NULL), set the head to the new node.
3. If the list is not empty, traverse the list to the last node.
4. Set the next pointer of the last node to the new node.
5. Set the prev pointer of the new node to the last node.

4. Delete from the Beginning of the List

Input: Pointer to the head of the list. Output: List with the first node deleted.

Steps:

1. If the list is empty (head is NULL), print a message and return.
2. Set a temporary pointer temp to the head.
3. Set the head to the next node of the current head.
4. If the head is not NULL, set its prev pointer to NULL.
5. Free the memory allocated for the node pointed to by temp.

5. Delete from the End of the List

Input: Pointer to the head of the list. Output: List with the last node deleted.

Steps:

1. If the list is empty (head is NULL), print a message and return.
2. If the list has only one node, set the head to NULL and free the memory.
3. Otherwise, traverse the list to the last node.
4. Set the next pointer of the second-last node to NULL.
5. Free the memory allocated for the last node.

6. Traverse the List in Forward Direction

Input: Pointer to the head of the list. Output: Prints the list in forward direction (head to last).

Steps:

1. Set a temporary pointer temp to the head.
2. While temp is not NULL:
   - Print the data of temp.
   - Move temp to the next node.
3. Stop when the end of the list is reached.

7. Traverse the List in Backward Direction

Input: Pointer to the head of the list. Output: Prints the list in backward direction (last to head).

Steps:

1. Set a temporary pointer temp to the head.
2. Traverse the list to the last node (move temp to the last node using the next pointer).
3. While temp is not NULL:
   - Print the data of temp.
   - Move temp to the previous node.
4. Stop when the start of the list is reached.

```
#include <stdio.h>
#include <stdlib.h>
```

ANSHID BABU T M

```
// Define the structure for a doubly linked list node
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

// Function to insert at the beginning of the doubly linked list
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        newNode->next = *head;
        (*head)->prev = newNode;
        *head = newNode;
    }
}

// Function to insert at the end of the doubly linked list
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        struct Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}

// Function to delete a node from the beginning
void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }
```

ANSHID BABU T M

```
    struct Node* temp = *head;
    *head = (*head)->next;
    if (*head != NULL) {
        (*head)->prev = NULL;
    }
    free(temp);
}

// Function to delete a node from the end
void deleteFromEnd(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    if (temp->prev != NULL) {
        temp->prev->next = NULL;
    } else {
        *head = NULL; // If there's only one node
    }
    free(temp);
}

// Function to print the list from beginning to end
void printForward(struct Node* head) {
    struct Node* temp = head;
    printf("List in Forward Direction: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Function to print the list from end to beginning
void printBackward(struct Node* head) {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty.\n");
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    printf("List in Backward Direction: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
```

ANSHID BABU T M

```
        temp = temp->prev;
    }
    printf("\n");
}

// Main function to test the doubly linked list implementation
int main() {
    struct Node* head = NULL;

    // Insert elements at the beginning and end
    insertAtBeginning(&head, 10);
    insertAtBeginning(&head, 20);
    insertAtEnd(&head, 5);
    insertAtEnd(&head, 15);

    // Print the list in both directions
    printForward(head);
    printBackward(head);

    // Delete from beginning and end
    deleteFromBeginning(&head);
    deleteFromEnd(&head);

    // Print the updated list
    printForward(head);
    printBackward(head);

    return 0;
}
```

```
List in Forward Direction: 20 10 5 15
List in Backward Direction: 15 5 10 20
List in Forward Direction: 10 5
List in Backward Direction: 5 10
```

25. Implement a binary search tree of characters.

Algorithm for Binary Search Tree

1. Creating a Node
Input: A character. Output: A new node containing the character.
Steps:
    1. Allocate memory for a new node.
    2. Set the data of the node to the given character.
    3. Set both left and right pointers of the node to NULL.
    4. Return the new node.
2. Insert a Character into the BST
Input: A character and a pointer to the root of the BST. Output: The BST with the new

ANSHID BABU T M

character inserted.
Steps:

1. If the tree is empty (i.e., the root is NULL), create a new node with the given character and set it as the root.
2. If the tree is not empty, compare the given character with the root's character:
    - If the character is smaller than the root's character, insert it into the left subtree.
    - If the character is greater than the root's character, insert it into the right subtree.
3. Continue the insertion recursively until the appropriate leaf node is found.

3. Search for a Character in the BST
Input: A character and a pointer to the root of the BST. Output: Return TRUE if the character is found, else FALSE.
Steps:

1. If the root is NULL, return FALSE (character not found).
2. If the character matches the root's character, return TRUE.
3. If the character is smaller than the root's character, search in the left subtree.
4. If the character is greater than the root's character, search in the right subtree.

4. Inorder Traversal (Left, Root, Right)
Input: A pointer to the root of the BST. Output: Print the characters in sorted order.
Steps:

1. Traverse the left subtree recursively.
2. Print the root node's character.
3. Traverse the right subtree recursively.

5. Delete a Character from the BST
Input: A character and a pointer to the root of the BST. Output: The BST with the character removed.
Steps:

1. If the tree is empty (root is NULL), return NULL (no element to delete).
2. If the character to be deleted is smaller than the root's character, delete it from the left subtree.
3. If the character to be deleted is greater than the root's character, delete it from the right subtree.
4. If the character matches the root's character, perform the following steps:
    - If the node has no children, set the node to NULL.
    - If the node has one child, replace the node with its child.
    - If the node has two children, find the in-order successor (smallest node in the right subtree), replace the current node's data with the in-order successor, and delete the in-order successor.

```
#include <stdio.h>
#include <stdlib.h>

// Structure for a node in the BST
struct Node {
   char data;
   struct Node* left;
   struct Node* right;
};
```

ANSHID BABU T M

```
// Function to create a new node
struct Node* createNode(char data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Function to insert a node into the BST
struct Node* insert(struct Node* root, char data) {
    if (root == NULL) {
        return createNode(data);
    }
    if (data < root->data) {
        root->left = insert(root->left, data);
    } else if (data > root->data) {
        root->right = insert(root->right, data);
    }
    return root;
}

// Function to search for a node in the BST
int search(struct Node* root, char key) {
    if (root == NULL) {
        return 0; // Not found
    }
    if (root->data == key) {
        return 1; // Found
    } else if (key < root->data) {
        return search(root->left, key);
    } else {
        return search(root->right, key);
    }
}

// Function for in-order traversal of the BST (Left, Root, Right)
void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%c ", root->data);
        inorderTraversal(root->right);
    }
}

// Function to find the minimum value node in the BST
struct Node* findMin(struct Node* root) {
    while (root && root->left != NULL) {
        root = root->left;
    }
    return root;
```

ANSHID BABU T M

```
}

// Function to delete a node from the BST
struct Node* delete(struct Node* root, char key) {
    if (root == NULL) {
        return root; // Element not found
    }

    // Traverse the tree
    if (key < root->data) {
        root->left = delete(root->left, key);
    } else if (key > root->data) {
        root->right = delete(root->right, key);
    } else {
        // Node with only one child or no child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }

        // Node with two children: Get the in-order successor
        struct Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = delete(root->right, temp->data);
    }
    return root;
}

// Main function
int main() {
    struct Node* root = NULL;
    root = insert(root, 'D');
    root = insert(root, 'B');
    root = insert(root, 'A');
    root = insert(root, 'C');
    root = insert(root, 'F');
    root = insert(root, 'E');
    root = insert(root, 'G');

    printf("In-order traversal of the BST:\n");
    inorderTraversal(root);
    printf("\n");

    char key = 'C';
    if (search(root, key)) {
```

ANSHID BABU T M

```
        printf("Character '%c' found in the tree.\n", key);
    } else {
        printf("Character '%c' not found in the tree.\n", key);
    }

    printf("Deleting character 'B' from the tree...\n");
    root = delete(root, 'B');
    inorderTraversal(root);
    printf("\n");

    return 0;
}
```

In-order traversal of the BST:
A B C D E F G
Character 'C' found in the tree.
Deleting character 'B' from the tree...
A C D E F G

ANSHID BABU T M